**UNIVERSIDAD PRIVADA DOMINGO SAVIO**
**Facultad de Ingeniería**
**Carrera de Ingeniería de Sistemas**

Desarrollo de Sistemas II

# Actividad Académica N° 2: Implementación CRUD bajo la arquitectura MVC

Sistema de Pedidos Mayoristas para Laboratorio Químico Delta
Módulos de Farmacias y Medicamentos

**Estudiante:** Gustavo Rodríguez Vélez
**Docente:** Vladimir Wilmar Rojas Condori

Cochabamba, 8 de noviembre de 2026

# Índice

Descripción de los módulos implementados
Configuración del entorno de desarrollo
Estructura del proyecto bajo MVC
Configuración de la conexión a la base de datos
Implementación de modelos
Implementación de controladores
Operaciones CRUD
Validación de datos
Manejo de errores y excepciones
Evidencias de funcionamiento
Repositorio GitHub
Conclusiones
Anexos

# Descripción de los módulos implementados

En esta actividad implementamos los dos primeros módulos del Sistema de Pedidos Mayoristas de Laboratorio Químico Delta, siguiendo el diseño presentado en la Actividad 1: el módulo de Farmacias y el módulo de Medicamentos. Elegimos estos dos porque son las entidades centrales del proceso de pedidos y tienen pocas dependencias. Un pedido no se puede registrar sin una farmacia a quien venderle ni sin medicamentos que ofrecer, así que tenerlos funcionando primero deja el camino listo para la Actividad 3.

El módulo de Farmacias permite registrar una farmacia cliente, ver el listado de farmacias activas con búsqueda por nombre o NIT, consultar el detalle, modificar sus datos y desactivarla. El módulo de Medicamentos hace lo mismo con el catálogo, mostrando la categoría de cada medicamento, su precio mayorista y su stock, con un filtro por categoría. La tabla categoria se usa solo como apoyo para el formulario de medicamentos, con las categorías que carga el script inicial.

Mantuvimos el mismo proyecto, el mismo problema y la misma base de datos de la Actividad 1. La tecnología sigue siendo la propuesta en la matriz tecnológica (Node.js, Express, EJS y MySQL 8), que todavía está sujeta a la aprobación docente.

## Matriz de trazabilidad

| Requerimiento | Entidad / tabla | Operación | Evidencia de cumplimiento |
|---|---|---|---|
| RF-06 Registrar farmacia | farmacia | CREATE | CP-01, CP-02, CP-03 y CP-04 |
| RF-07 Mostrar el listado de farmacias activas | farmacia | READ | CP-05 |
| RF-08 Mostrar el detalle de una farmacia | farmacia | READ | CP-05 y CP-06 |
| RF-09 Modificar los datos de una farmacia | farmacia | UPDATE | CP-07 y CP-08 |
| RF-10 Desactivar una farmacia | farmacia | DELETE lógico | CP-09 |
| RF-13 Registrar un medicamento | medicamento, categoria | CREATE | CP-10, CP-11, CP-12 y CP-14 |
| RF-14 Mostrar el listado de medicamentos | medicamento, categoria | READ | CP-10 |
| RF-15 Mostrar el detalle de un medicamento | medicamento | READ | CP-13 y CP-15 |
| RF-16 Modificar los datos de un medicamento | medicamento | UPDATE | CP-13 |
| RF-17 Desactivar un medicamento | medicamento | DELETE lógico | CP-15 |

La búsqueda de farmacias por nombre o NIT (RF-11), que en la Actividad 1 quedó para la Actividad 3, se adelantó porque era sencilla de agregar al listado.

## Configuración del entorno de desarrollo

Trabajamos en Windows 11 con las siguientes herramientas.

| Herramienta | Versión | Función |
|---|---|---|
| Visual Studio Code | Actual | Editor del código y la documentación. |
| Node.js | 26.13.0 | Entorno de ejecución del backend. |
| npm | 11.20.0 | Instalación de dependencias. |
| Express | 4.22.3 | Framework para las rutas y los controladores. |
| EJS | 7.0.2 | Motor de vistas. |
| mysql2 | 3.24.6 | Conexión y consultas parametrizadas a MySQL. |
| dotenv | 18.0.9 | Lectura de las variables del archivo .env. |
| MySQL Community Server | 8.4.9 | Motor de base de datos. |
| MySQL Workbench | 8.0 | Gestor gráfico de la base de datos. |
| Git | 2.56.0 | Control de versiones. |
| Navegador web (Chrome) | Actual | Pruebas de las pantallas y las rutas. |

Las versiones las verificamos desde la terminal:

```
$ git --version
git version 2.56.0.windows.4
$ node --version
v26.13.0
$ npm --version
11.20.0
$ mysql --version
C:\Program Files\MySQL\MySQL Server 8.4\bin\mysql.exe  Ver 8.4.9 for Win64 on
x86_64 (MySQL Community Server - GPL)
```

Para preparar el proyecto se instalaron las dependencias con npm install, se creó la base de datos ejecutando el script database/script_inicial.sql y se copió el archivo .env.example como .env, completando la contraseña del usuario de la base. La aplicación se inicia con npm start y queda disponible en http://localhost:3000.

## Estructura del proyecto bajo MVC

El proyecto sigue la organización que planteamos en el diagrama de componentes de la Actividad 1. Cada parte tiene una responsabilidad concreta y no se mezclan las consultas SQL con la presentación.

```
sistema-pedidos-delta/
  database/script_inicial.sql      Script de creación de la base
  public/css/estilos.css           Estilos (los mismos del prototipo)
```

```
  src/
    app.js                          Punto de entrada: configura Express y arranca el
servidor
    config/db.js                    Conexión a MySQL (grupo de conexiones)
    models/                         farmaciaModel.js, medicamentoModel.js,
categoriaModel.js
    controllers/                    farmaciaController.js, medicamentoController.js
    routes/                         farmaciaRoutes.js, medicamentoRoutes.js
    validators/                     farmaciaValidator.js, medicamentoValidator.js
    views/                          farmacias/, medicamentos/, errores/, partials/
  .env.example                      Variables de entorno de ejemplo (sin
contraseñas)
```

| Parte | Responsabilidad | Ejemplo en el módulo Farmacias |
|---|---|---|
| Modelo | Consultar y modificar la tabla correspondiente. Es la única parte que habla con la base de datos. | farmaciaModel.js con listarActivas, buscarPorId, crear, actualizar y desactivar. |
| Controlador | Recibir la solicitud, validar los datos, llamar al modelo y decidir qué vista mostrar o a dónde redirigir. | farmaciaController.js con las acciones listar, crear, detalle, actualizar y desactivar. |
| Vista | Mostrar el listado, el detalle, los formularios y los mensajes. | views/farmacias/index.ejs, form.ejs y detalle.ejs. |
| Rutas | Relacionar cada dirección y método HTTP con una acción del controlador. | farmaciaRoutes.js. |
| Validación | Revisar los datos que llegan del formulario antes de guardarlos. | farmaciaValidator.js. |
| Configuración | Parámetros de conexión leídos del archivo .env. | config/db.js. |

El recorrido de una solicitud es el siguiente. Cuando el usuario envía el formulario de nueva farmacia, el navegador hace un POST a /farmacias. Express busca esa ruta en farmaciaRoutes.js y llama a la acción crear del controlador. El controlador pasa los datos al validador; si hay errores vuelve a mostrar el formulario con los mensajes. Si los datos son correctos, llama a Farmacia.crear, que ejecuta un INSERT con parámetros sobre la tabla farmacia. Cuando MySQL confirma la inserción, el controlador redirige a GET /farmacias, donde el modelo consulta las farmacias activas y la vista index.ejs arma la página con el mensaje de éxito que vuelve al navegador.

## Configuración de la conexión a la base de datos

La conexión se configura en src/config/db.js con un grupo de conexiones de mysql2. Los datos de acceso no están escritos en el código: se leen de variables de entorno cargadas desde el archivo .env, que está excluido del repositorio en el .gitignore. En el repositorio solo se sube .env.example, con las mismas variables pero sin la contraseña.

```
PORT=3000
DB_HOST=127.0.0.1
DB_PORT=3306
```

```
DB_USER=delta_app
DB_PASSWORD=(oculta)
DB_NAME=delta_pedidos
```

La aplicación no usa la cuenta root. Creamos un usuario delta_app con permisos solo sobre la base delta_pedidos. Al iniciar, app.js ejecuta una consulta de prueba; si la conexión funciona muestra el nombre de la base, y si falla muestra un mensaje claro y detiene el servidor en lugar de quedar funcionando sin base de datos.

```
$ PORT=3001 npm start   (inicio normal)
Conexión a MySQL establecida (base de datos: delta_pedidos)
Servidor iniciado en http://localhost:3001

$ npm start   (con otra instancia ya usando el puerto 3000)
Conexión a MySQL establecida (base de datos: delta_pedidos)
El puerto 3000 ya está en uso. Cierre la otra instancia o cambie PORT en el archivo
.env.

$ DB_PASSWORD=incorrecta npm start   (contraseña de base de datos incorrecta)
No se pudo conectar a MySQL. Revise los datos del archivo .env.
Detalle: ER_ACCESS_DENIED_ERROR

$ git check-ignore -v .env
.gitignore:2:.env   .env
```

### Tablas utilizadas

La base se creó con el script database/script_inicial.sql de la Actividad 1, sin cambios. El script crea las siete tablas del modelo (rol, usuario, farmacia, categoria, medicamento, pedido y detalle_pedido) y carga datos ficticios de prueba. En esta actividad se usan farmacia, medicamento y categoria; las demás quedan listas para los módulos de pedidos y usuarios.

| Modelo físico (Actividad 1) | Estructura creada en MySQL | Uso en el código |
|---|---|---|
| farmacia (id_farmacia, nombre, nit UNIQUE, responsable, telefono, direccion, zona, activo, fecha_registro) | Igual, con InnoDB y utf8mb4 | Mismos nombres de columnas en farmaciaModel.js |
| categoria (id_categoria, nombre UNIQUE, descripcion) | Igual | Solo lectura en categoriaModel.js |
| medicamento (id_medicamento, nombre, presentacion, id_categoria FK, precio_mayorista CHECK > 0, stock CHECK >= 0, activo, UNIQUE nombre y presentación) | Igual | Mismos nombres de columnas en medicamentoModel.js |

No fue necesario cambiar el modelo aprobado. El campo activo, que ya estaba en el diseño, es el que permite la desactivación lógica.

## Implementación de modelos

Cada modelo es un objeto con funciones asíncronas que reciben datos simples y devuelven filas o la cantidad de filas afectadas. Todas las consultas usan marcadores de posición (?) y los valores se pasan aparte, de modo que el texto que escribe el usuario nunca se concatena dentro del SQL. Por ejemplo, el modelo de farmacias registra una farmacia con:

```
INSERT INTO farmacia (nombre, nit, responsable, telefono, direccion, zona)
VALUES (?, ?, ?, ?, ?, ?)
```

Las consultas de listado filtran por activo = 1 para no mostrar los registros desactivados. El modelo de medicamentos une medicamento con categoria para mostrar el nombre de la categoría sin repetirlo en la tabla, tal como se planteó en el modelo lógico. Las actualizaciones incluyen la condición AND activo = 1, así un registro desactivado no se puede modificar por accidente.

## Implementación de controladores

Los controladores farmaciaController.js y medicamentoController.js tienen una acción por operación. Cada acción convierte el parámetro :id a número y, si no es un entero válido, responde directamente con la página de no encontrado. Luego valida los datos, llama al modelo y, según el resultado, redirige con un mensaje de éxito (por ejemplo /farmacias?ok=registrada), vuelve a mostrar el formulario con los errores o muestra la página de no encontrado. Después de guardar se redirige en lugar de mostrar la vista directamente, para que al recargar la página el navegador no vuelva a enviar el formulario.

## Operaciones CRUD

Crear: los formularios de nueva farmacia y nuevo medicamento envían los datos por POST. El controlador valida, guarda y redirige al listado, donde aparece el mensaje de confirmación. Comprobamos en MySQL que cada registro realmente quedó guardado (CP-01 y CP-10).

Consultar: los listados muestran solo los registros activos, ordenados por nombre, con búsqueda por nombre o NIT en farmacias y filtro por categoría en medicamentos. Cuando no hay resultados se muestra un aviso en lugar de una tabla vacía. El detalle se consulta por identificador y, si no existe o está desactivado, se responde con código 404 y un mensaje entendible (CP-05 y CP-06).

Actualizar: el formulario de edición carga los valores actuales. El identificador no se puede cambiar porque va en la ruta y no en el formulario. Los nuevos datos pasan por las mismas validaciones del registro y, al guardar, se muestra el detalle actualizado. Verificamos el cambio volviendo a consultar la tabla (CP-07 y CP-13).

Eliminar: decidimos usar desactivación lógica y no borrado físico. Una farmacia o un medicamento pueden aparecer en pedidos anteriores, y si se borraran se perdería ese

historial o la base rechazaría el borrado por las claves foráneas. Por eso la acción de desactivar cambia activo a 0, previa confirmación del usuario. El registro desaparece de los listados y de los formularios, pero se conserva en la tabla (CP-09 y CP-15).

### Rutas implementadas

Como las vistas se generan en el servidor, las operaciones se exponen como rutas MVC que reciben formularios. Los formularios HTML solo envían GET y POST, por eso la edición y la desactivación usan POST con una ruta propia.

| Operación | Método | Ruta | Resultado esperado |
|---|---|---|---|
| Listar farmacias | GET | /farmacias | Listado de farmacias activas (acepta ? buscar=). |
| Formulario de registro | GET | /farmacias/nueva | Formulario vacío. |
| Registrar farmacia | POST | /farmacias | Guarda y redirige al listado, o muestra errores. |
| Consultar una farmacia | GET | /farmacias/:id | Detalle de la farmacia o 404. |
| Formulario de edición | GET | /farmacias/:id/editar | Formulario con los datos actuales. |
| Actualizar farmacia | POST | /farmacias/:id/editar | Guarda y muestra el detalle, o muestra errores. |
| Desactivar farmacia | POST | /farmacias/:id/desactivar | Cambia activo a 0 y redirige al listado. |
| Listar medicamentos | GET | /medicamentos | Catálogo activo (acepta ?categoria=). |
| Formulario de registro | GET | /medicamentos/nuevo | Formulario con las categorías. |
| Registrar medicamento | POST | /medicamentos | Guarda y redirige al catálogo, o muestra errores. |
| Consultar un medicamento | GET | /medicamentos/:id | Detalle del medicamento o 404. |
| Formulario de edición | GET | /medicamentos/:id/editar | Formulario con los datos actuales. |
| Actualizar medicamento | POST | /medicamentos/:id/editar | Guarda y muestra el detalle, o muestra errores. |
| Desactivar medicamento | POST | /medicamentos/:id/desactivar | Cambia activo a 0 y redirige al catálogo. |

## Validación de datos

Las validaciones se hacen en el servidor, en los archivos de la carpeta validators. Los formularios usan el atributo novalidate para que las reglas del navegador no oculten las del backend, ya que no conviene confiar solo en lo que valida el frontend. Además, la base de datos mantiene sus propias restricciones (NOT NULL, UNIQUE, CHECK y claves foráneas) como segunda barrera.

| Campo | Regla | Mensaje |
|---|---|---|
| farmacia.nombre | Obligatorio, máximo 100 caracteres | El nombre es obligatorio. |

| Campo | Regla | Mensaje |
|---|---|---|
| farmacia.nit | Obligatorio, solo números, de 7 a 20 dígitos, único | El NIT debe tener solo números (entre 7 y 20 dígitos). / Ya existe una farmacia registrada con este NIT. |
| farmacia.telefono | Obligatorio, números, espacios, + o -, de 7 a 20 caracteres | El teléfono es obligatorio. |
| farmacia.direccion | Obligatoria, máximo 150 caracteres | La dirección es obligatoria. |
| farmacia.zona | Obligatoria y dentro de la lista de zonas | Seleccione una zona. |
| medicamento.nombre y presentacion | Obligatorios; la combinación no se puede repetir | Ya existe un medicamento con el mismo nombre y presentación. |
| medicamento.id_categoria | Obligatoria y debe existir en la tabla categoria | La categoría seleccionada no existe. |
| medicamento.precio_mayorista | Número con hasta dos decimales, mayor que cero | El precio debe ser mayor que cero. |
| medicamento.stock | Número entero, igual o mayor que cero | El stock no puede ser negativo. |

Cuando hay errores, el formulario se vuelve a mostrar con los datos que el usuario ya escribió, el campo marcado en rojo y el mensaje debajo, para que solo tenga que corregir lo que falta.

# Manejo de errores y excepciones

Controlamos los siguientes casos:

| Situación | Cómo se maneja |
|---|---|
| Registro no encontrado o identificador con formato inválido | Página de no encontrado con código 404 y enlace para volver. |
| Datos inválidos | El formulario se muestra de nuevo con código 422 y los mensajes por campo; no se guarda nada. |
| NIT o medicamento duplicado | Se captura el error ER_DUP_ENTRY de MySQL y se muestra un mensaje en el campo, con código 409. |
| Error inesperado del servidor o de la base | Un manejador central registra el detalle técnico en la consola y al usuario solo le muestra un mensaje general, sin trazas ni consultas SQL. |
| Fallo de conexión al iniciar | Mensaje claro en la terminal y el servidor no arranca. |
| Puerto ocupado | Mensaje claro en la terminal en lugar de la traza completa (corregido durante las pruebas). |
| Dirección inexistente | Página de no encontrado con código 404. |

## Medidas básicas de seguridad

Los controles que existen en esta versión son: consultas parametrizadas en todos los modelos, credenciales en un archivo .env fuera del repositorio, un usuario de base de datos sin privilegios de administrador, validación en el servidor, mensajes de error que no revelan detalles internos y datos ficticios en todas las pruebas. Las vistas EJS escapan el texto que se muestra, lo que evita que un nombre con etiquetas HTML se ejecute en la página.

Queremos ser claros en que la seguridad todavía no está completa. En esta versión no hay inicio de sesión ni control por roles: cualquiera que acceda a la aplicación en el equipo local puede usar los dos módulos. La autenticación, las contraseñas cifradas y la restricción por rol (RF-01 a RF-05 y RNF-01, RNF-02) están planificadas para la Actividad 3.

# Evidencias de funcionamiento

## Casos de prueba

Las pruebas se ejecutaron sobre la aplicación funcionando con la base delta_pedidos recién creada con el script inicial. Para no depender de la memoria, automatizamos el recorrido en el navegador con Playwright: el script llena los formularios igual que lo haría un usuario, guarda una captura de cada paso y anota la respuesta real que dio el sistema. La columna de resultado obtenido muestra lo que efectivamente respondió la aplicación.

| Código | Caso de prueba | Resultado esperado | Resultado obtenido | Estado | Captura |
|---|---|---|---|---|---|
| CP-01 | Registrar una farmacia con todos los datos válidos. | Se guarda la farmacia, se muestra un mensaje de confirmación y aparece en el listado. | Mensaje "La farmacia se registró correctamente.". La consulta a MySQL devuelve la fila con id 5. | Aprobado | 01-cp01-formulario-farmacia, 02-cp01-farmacia-registrada |
| CP-02 | Registrar una farmacia sin NIT, teléfono, dirección ni zona. | El sistema rechaza el registro y muestra qué campos faltan. | Se mostraron 4 mensajes: El NIT es obligatorio. El teléfono es obligatorio. La dirección es obligatoria. Seleccione una zona. No se guardó ninguna fila. | Aprobado | 03-cp02-campos-obligatorios |
| CP-03 | Registrar una farmacia con un NIT que ya existe. | El sistema informa que el NIT ya está registrado y no guarda el registro. | Mensaje "Ya existe una farmacia registrada con este NIT.". | Aprobado | 04-cp03-nit-duplicado |

| Código | Caso de prueba | Resultado esperado | Resultado obtenido | Estado | Captura |
|---|---|---|---|---|---|
| CP-04 | Registrar una farmacia con NIT "12AB" y teléfono "abc". | El sistema rechaza los formatos incorrectos con mensajes claros. | Mensajes: El NIT debe tener solo números (entre 7 y 20 dígitos). El teléfono solo puede tener números, espacios, + o - (de 7 a 20 caracteres). | Aprobado | 05-cp04-formato-invalido |
| CP-05 | Consultar el listado de farmacias y el detalle de la farmacia registrada. | Se muestran los datos guardados en la base de datos. | El listado muestra 5 farmacias activas y el detalle muestra "Farmacia San Martín de Prueba". | Aprobado | 06-cp05-listado-farmacias, 07-cp05-detalle-farmacia |
| CP-06 | Consultar la farmacia con id 9999, que no existe. | Se informa que el registro no existe, sin mostrar errores técnicos. | Código HTTP 404 y mensaje "La farmacia que busca no existe o fue desactivada.". | Aprobado | 08-cp06-farmacia-inexistente |
| CP-07 | Cambiar el teléfono y la zona de la farmacia registrada. | Los cambios se guardan y se ven al consultar de nuevo. | Mensaje "Los datos de la farmacia se actualizaron correctamente.". En MySQL el teléfono quedó 70000077 y la zona Sacaba. | Aprobado | 09-cp07-editar-farmacia, 10-cp07-farmacia-actualizada |
| CP-08 | Enviar una actualización para la farmacia con id 9999. | Se responde que el registro no existe. | Código HTTP 404, página de "Farmacia no encontrada". | Aprobado | 11-cp08-actualizar-inexistente |
| CP-09 | Desactivar la farmacia registrada desde el listado. | La farmacia deja de aparecer en el listado y en la base queda con activo = 0. | Mensaje "La farmacia fue desactivada. Su historial de pedidos se conserva.". El NIT ya no aparece en el listado y la fila conserva activo = 0. | Aprobado | 12-cp09-farmacia-desactivada |
| CP-10 | Registrar un medicamento con datos válidos. | Se guarda el medicamento con su categoría y aparece en el catálogo. | Mensaje "El medicamento se registró correctamente.". MySQL devuelve el medicamento con id 7, categoría Antialérgicos y precio 33.50. | Aprobado | 13-cp10-formulario-medicamento, 14-cp10-medicamento-registrado |
| CP-11 | Registrar un medicamento con precio -15 y stock -3. | El sistema rechaza los valores y explica la regla. | Mensajes: El precio debe ser mayor que cero. El stock no puede ser negativo. | Aprobado | 15-cp11-precio-stock-negativos |
| CP-12 | Enviar un medicamento con una categoría que no existe (id 99), alterando el formulario desde el navegador. | El backend vuelve a validar y rechaza la categoría. | Mensaje "La categoría seleccionada no existe.". | Aprobado | 16-cp12-categoria-inexistente |

| Código | Caso de prueba | Resultado esperado | Resultado obtenido | Estado | Captura |
|---|---|---|---|---|---|
| CP-13 | Cambiar el precio a 35.90 y el stock a 150 del medicamento registrado. | Los nuevos valores quedan guardados. | Mensaje "Los datos del medicamento se actualizaron correctamente.". MySQL muestra precio 35.90 y stock 150. | Aprobado | 17-cp13-medicamento-actualizado |
| CP-14 | Registrar otra vez Cetirizina 10 mg en Caja x 100 tabletas. | Se informa que ya existe un medicamento con ese nombre y presentación. | Mensaje "Ya existe un medicamento con el mismo nombre y presentación.". | Aprobado | 18-cp14-medicamento-duplicado |
| CP-15 | Desactivar el medicamento registrado desde su detalle. | Deja de aparecer en el catálogo y queda con activo = 0. | Mensaje "El medicamento fue desactivado y ya no aparece en el catálogo.". MySQL muestra activo = 0 y su detalle ahora responde 404. | Aprobado | 19-cp15-medicamento-desactivado |

## Recorrido CREATE, READ, UPDATE y DELETE

Los casos CP-01, CP-05, CP-07 y CP-09 forman el recorrido completo sobre una misma farmacia: se registra, aparece en el listado y en su detalle, se modifica y finalmente se desactiva. Lo mismo ocurre con un medicamento en CP-10, CP-13 y CP-15. En cada paso consultamos la tabla en MySQL para confirmar que el cambio quedó guardado y que no quedaron datos inconsistentes. Estas son las consultas principales, ejecutadas con el usuario de la aplicación (el resto está en evidencias/resultados.json):

CP-01: la farmacia registrada existe en la base de datos:

```
mysql> SELECT id_farmacia, nombre, nit, telefono, zona, activo FROM farmacia WHERE
nit = '1000000055';
+-------------+--------------------------------+------------+----------+-------+---
-----+
| id_farmacia | nombre                         | nit        | telefono | zona  |
activo |
+-------------+--------------------------------+------------+----------+-------+---
-----+
|           5 | Farmacia San Martín de Prueba  | 1000000055 | 70000005 | Norte |
1 |
+-------------+--------------------------------+------------+----------+-------+---
-----+
```

CP-07: valor antes de la actualización:

```
mysql> SELECT id_farmacia, nombre, telefono, zona FROM farmacia WHERE id_farmacia =
5;
+-------------+--------------------------------+----------+-------+
| id_farmacia | nombre                         | telefono | zona  |
```

```
+-------------+--------------------------------+----------+-------+
|           5 | Farmacia San Martín de Prueba  | 70000005 | Norte |
+-------------+--------------------------------+----------+-------+
```

CP-07: valor después de la actualización:

```
mysql> SELECT id_farmacia, nombre, telefono, zona FROM farmacia WHERE id_farmacia =
5;
+-------------+--------------------------------+----------+--------+
| id_farmacia | nombre                         | telefono | zona   |
+-------------+--------------------------------+----------+--------+
|           5 | Farmacia San Martín de Prueba  | 70000077 | Sacaba |
+-------------+--------------------------------+----------+--------+
```

CP-09: la farmacia desactivada se conserva con activo = 0:

```
mysql> SELECT id_farmacia, nombre, activo FROM farmacia WHERE id_farmacia = 5;
+-------------+--------------------------------+--------+
| id_farmacia | nombre                         | activo |
+-------------+--------------------------------+--------+
|           5 | Farmacia San Martín de Prueba  |      0 |
+-------------+--------------------------------+--------+
```

CP-13: medicamento después de la actualización:

```
mysql> SELECT id_medicamento, nombre, precio_mayorista, stock FROM medicamento
WHERE id_medicamento = 7;
+----------------+------------------+------------------+-------+
| id_medicamento | nombre           | precio_mayorista | stock |
+----------------+------------------+------------------+-------+
|              7 | Cetirizina 10 mg |            35.90 |   150 |
+----------------+------------------+------------------+-------+
```

CP-15: el medicamento desactivado se conserva con activo = 0:

```
mysql> SELECT id_medicamento, nombre, activo FROM medicamento WHERE id_medicamento
= 7;
+----------------+------------------+--------+
| id_medicamento | nombre           | activo |
+----------------+------------------+--------+
|              7 | Cetirizina 10 mg |      0 |
+----------------+------------------+--------+
```

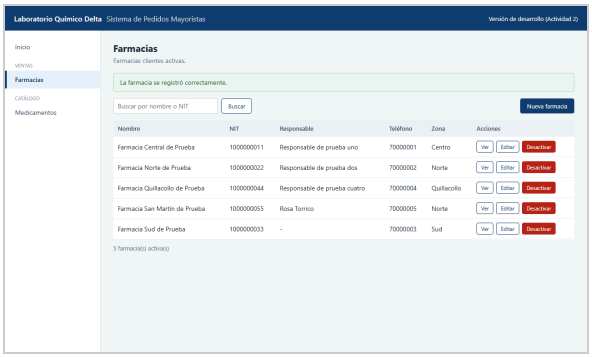
## Errores encontrados y corregidos

| Error | Causa | Corrección | Prueba repetida |
|---|---|---|---|
| Al iniciar la aplicación con el puerto 3000 ocupado, la terminal mostraba la traza completa de Node.js. | No se manejaba el evento de error del servidor al escuchar en el puerto. | Se agregó un manejador en app.js que muestra "El puerto 3000 ya está en uso..." y termina el proceso. | Se volvió a iniciar con el puerto ocupado y se obtuvo el mensaje esperado. |

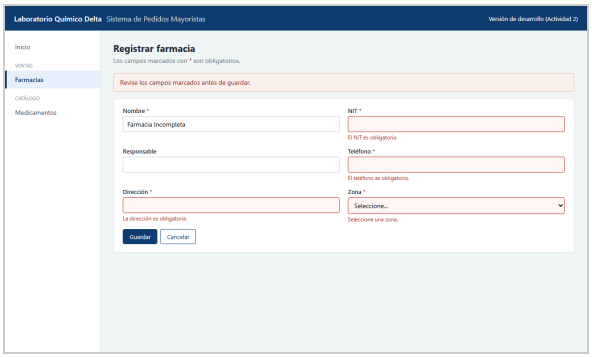
| Error | Causa | Corrección | Prueba repetida |
|---|---|---|---|
| La validación del precio aceptaba textos como "-abc" sin mostrar error (detectado al revisar el código antes de ejecutar las pruebas). | La expresión regular dejaba pasar cualquier valor que empezara con un signo menos. | Se cambió la expresión para aceptar solo números, con signo opcional, y luego comprobar que sea mayor que cero. | CP-11 rechaza el precio negativo con el mensaje correcto. |

## Capturas

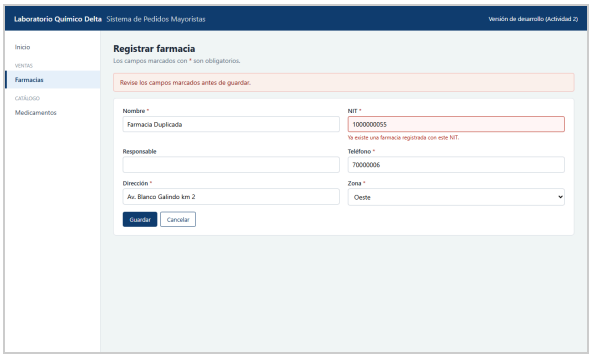
Se incluye la captura que muestra el resultado de los casos principales. Las 19 capturas completas, con los formularios llenos antes de enviar, están en la carpeta evidencias/capturas/ del repositorio.



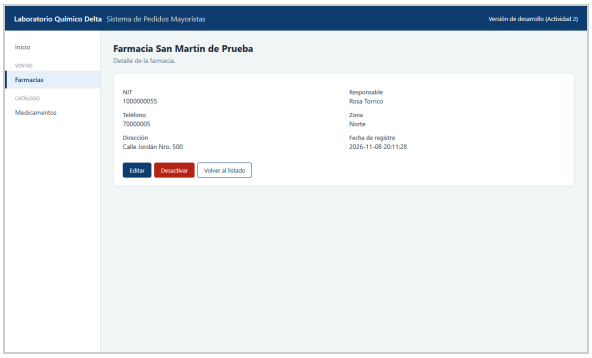

*CP-01: Registrar una farmacia con todos los datos válidos (02-cp01-farmacia-registrada)*



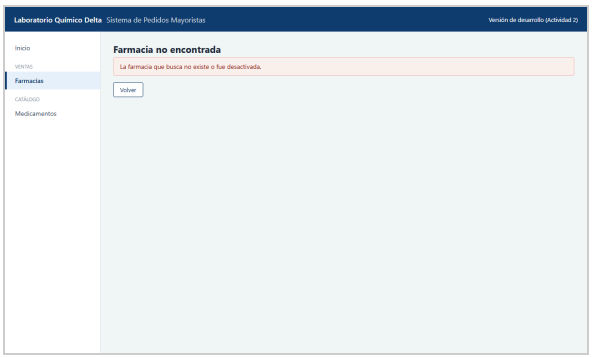

*CP-02: Registrar una farmacia sin NIT, teléfono, dirección ni zona (03-cp02-campos-obligatorios)*



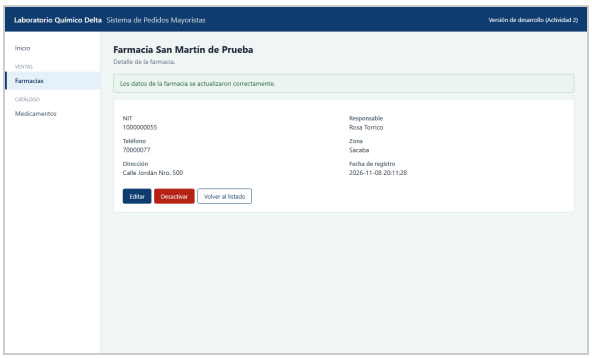

*CP-03: Registrar una farmacia con un NIT que ya existe (04-cp03-nit-duplicado)*



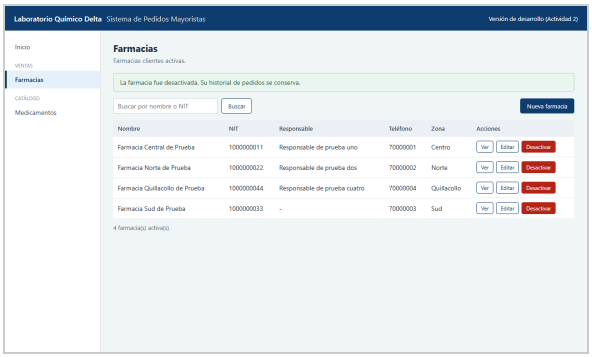

*CP-05: Consultar el listado de farmacias y el detalle de la farmacia registrada (07-cp05-detalle-farmacia)*



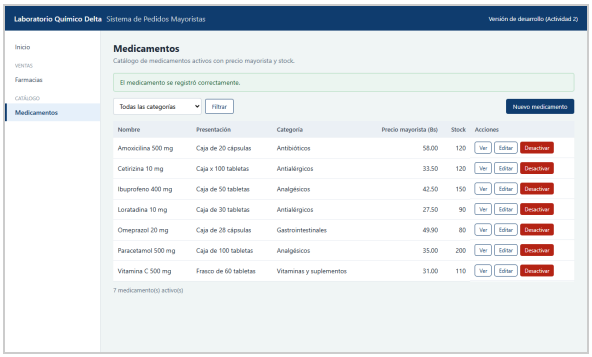

*CP-06: Consultar la farmacia con id 9999, que no existe (08-cp06-farmacia-inexistente)*



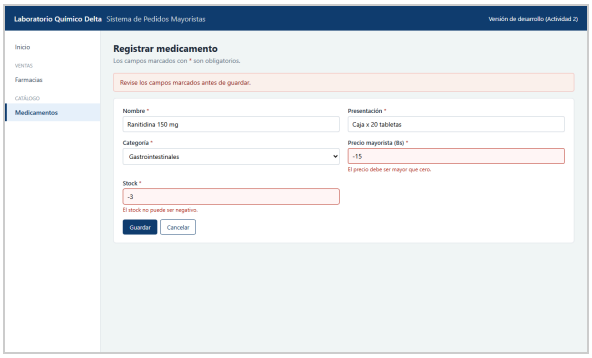

*CP-07: Cambiar el teléfono y la zona de la farmacia registrada (10-cp07-farmacia-actualizada)*



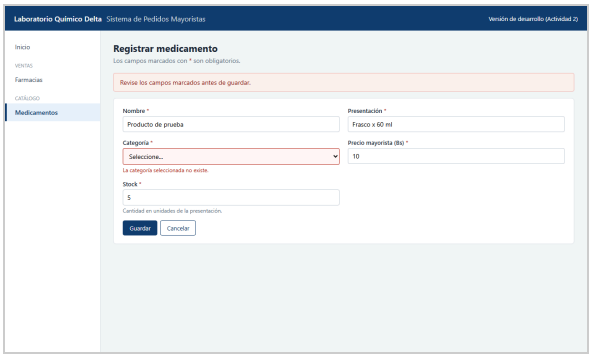

*CP-09: Desactivar la farmacia registrada desde el listado (12-cp09-farmacia-desactivada)*



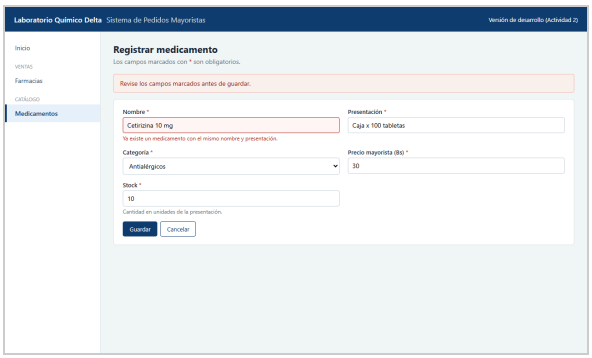

*CP-10: Registrar un medicamento con datos válidos (14-cp10-medicamento-registrado)*



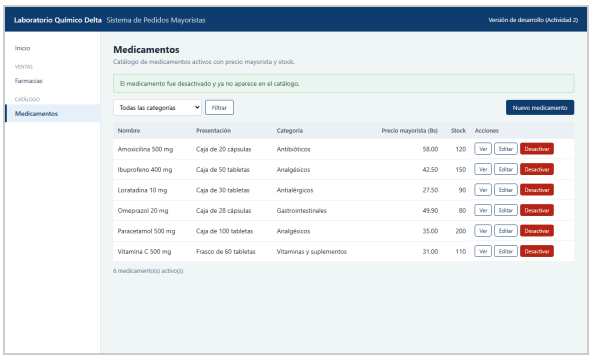

*CP-11: Registrar un medicamento con precio -15 y stock -3 (15-cp11-precio-stock-negativos)*




*CP-12: Enviar un medicamento con una categoría que no existe (id 99), alterando el formulario desde el navegador (16-cp12-categoria-inexistente)*




*CP-14: Registrar otra vez Cetirizina 10 mg en Caja x 100 tabletas (18-cp14-medicamento-duplicado)*




*CP-15: Desactivar el medicamento registrado desde su detalle (19-cp15-medicamento-desactivado)*

## Repositorio GitHub

El repositorio del proyecto se actualizó con el código de esta actividad:

https://github.com/GustavoRodVe/sistema-pedidos-delta

Además de la documentación, los diagramas y el prototipo de la Actividad 1, ahora contiene el código fuente en src/, los estilos en public/, el script de la base en database/, el archivo .env.example y la carpeta evidencias/ con las capturas y los resultados de las pruebas. Antes de cada commit revisamos con git status que no se incluyera el archivo .env ni la carpeta node_modules. Los commits de esta etapa describen cada avance, por ejemplo "Implementa la estructura MVC con Express y el CRUD de farmacias y medicamentos" y "Agrega evidencias de las pruebas del CRUD de farmacias y medicamentos".

## Conclusiones

Los módulos de Farmacias y Medicamentos quedaron funcionando de principio a fin: se pueden registrar, consultar, modificar y desactivar registros, y cada operación se comprobó contra la base de datos. Los 15 casos de prueba dieron el resultado esperado. Las dos fallas que encontramos en el camino, la del puerto ocupado y la del precio con texto inválido, se corrigieron y se volvieron a probar.

Separar el código en modelos, controladores, vistas, rutas y validadores hizo que cada archivo sea corto y fácil de seguir. Si cambia una regla de validación solo se toca el validador, y si cambia una consulta solo se toca el modelo. También comprobamos que haber definido bien la base en la Actividad 1 ahorró trabajo: el script se ejecutó sin cambios y los nombres de las columnas se usaron tal cual en el código.

Como limitaciones, esta versión no tiene inicio de sesión ni control por roles y las vistas son básicas. En la Actividad 3 continuaremos con la autenticación, el módulo de pedidos

con su detalle, el descuento de stock y la construcción completa del frontend a partir del prototipo, siempre sobre este mismo backend.

# Anexos

Anexo A. Código fuente completo en el repositorio (carpeta src/).

Anexo B. Script de la base de datos (database/script_inicial.sql).

Anexo C. Resultados de las pruebas en formato JSON y capturas originales (carpeta evidencias/).

Anexo D. Salida completa de la terminal usada como evidencia (evidencias/terminal.txt).